



3 📖

15 📄

1800 sp

نظري

كلية الهندسة المعلوماتية

السنة الخامسة - هندسة البرمجيات



Structural & Behavioral Pattern

د. باسم قصيبة

RB Informatcs;

21/10/2024

هندسة برمجيات ٣

تحدثنا في المحاضرة السابقة عن Creational Pattern و Structural Pattern وفي هذه المحاضرة سنكمل Structural Pattern و بعدها نبدأ ب Behavioral Pattern.

مثال: Debugger باستخدام ال Singleton و Decorator

الحالة الأولى: نقوم بعملية الطباعة من أجل كل عملية بالبرنامج كما هو موضح بالكود التالي:

```
public class BadSolution
{
    public static void main(String[] args)
    {
        System.out.println("Begin of program.");
        System.out.println("Begin of For.");
        for(int i=0; i<10; i++)
        {
            System.out.println("Begin of step i=" + i);
            // ...
            System.out.println("End of step i=" + i);
        }
        System.out.println("End of For.");
        //
        // Several others thousands of System.out.println().
        //
        System.out.println("End of program.");
    }
}
```

الحل السابق سيء لسببين:

1- إزالة عملية التنقيح (Debugging) بعد الانتهاء سيحتاج إلى تعديل وحذف في الكود.

2- بحال أردنا تغيير شكل الطباعة من ال console إلى ملف منفصل، أو أردنا طباعة الوقت أيضاً سنحتاج إلى تعديل الكود.

وكلا هذين الأمرين يخالف مبدأ ال OCP.

استخدام الواجهة Debugger:

- الهدف: فصل الاستخدام عن كيفية التنفيذ.
- يمكن إضافة وظائف جديدة عند الحاجة
- تحقيق ذلك برمجياً:

نعرف Debugger ك Interface ضمنه نعرف ال display من أجل توجيه ال Debuggers، ويتم تنفيذ هذه الواجهة في عدة صفوف: واحد للطباعة على الشاشة و الثاني في ملف و الثالث بدون طباعة:



```
public interface Debugger
{
    public void display(String message);
}
```

```
public class DebuggerToScreen implements Debugger {
    public void display(String message)
    {
        System.out.println(message);
    }
}
```

```
public class WithoutDebugger implements
Debugger {
    public void display(String message)
    { //nothing .... }
}
```

```
public class DebuggerToFile implements Debugger {
    public DebuggerToFile(String nameFile) throws
FileNotFoundException {
        FileOutputStream f = new FileOutputStream (nameFile);
        sf = new PrintStream (f);
    }
    public void display(String message)
    {sf.println(message);}
}
```

ضمن الـ Main :

نمرر نوع الـ debugger إلى تابع الـ Main ونقوم باستدعاء الـ display ضمن التابع السابق بما يتوافق مع نمط الـ debugger الذي تم تمريره لهذا التابع ، في حال صحة البرنامج بالكامل يمكن استدعاء التابع display بالنسبة للنمط Without Debugger (يقوم بتشغيل البرنامج دون أي طباعة على الشاشة).

```
public class SolutionWithDebugger {
    public static void main(String[] args) throws Exception {
        main("debugger to screen", new DebuggerToScreen());
        main("debugger to file", new DebuggerToFile("dbg.txt"));
        main("debugger in window", new DebuggerInWindow());
        main("without debugger", new WithoutDebugger());
    }
    public static void main(String title, Debugger dbg) {
        S.o.p(« Execution with " + title + ".");
        main(dbg);
    }
    public static void main(dbg) {
        dbg.display("Begin of program.");
        dbg.display("Begin of For.");
        for(int i=0; i<10; i++) {
            dbg.display("Begin of step i=" + i);
            // ...
            dbg.display("End of step i=" + i);
        }
        dbg.display("End of For");
        //....
        dbg.display("End of program.");
    }
}
```

رغم هذه التحسينات إلى أن هذا الحل يعد سيئاً وذلك لأننا بحاجة إلى نقل المنقح ضمن كل البرنامج وتمريضه كوسيط لكل الـ methods.

طريقة الـ Singleton: تقوم على بناء غرض واحد على طول البرنامج.

- ضمن كلاس Singleton Debugger قمنا بتعريف الـ getInstance() للحصول على الـ instance من نمط Debugger.
- وعرفنا أيضاً التابع setInstance() نمرر له الـ debugger كبرامتر وقمنا بإسناده إلى الـ Instance.
- وعرفنا التابع display لتطبيقه على الـ debugger العائد من التابع getInstance كما هو موضح بالكود التالي:

```
public abstract class SingletonDeboggeur {
    protected static Debugger instance = null;
    public static Debugger getInstance()
    {
        if(instance == null)
            instance = new
                WithoutDebugger();
        return instance;
    }
    public static void setInstance(Debugger dbg)
    {
        instance = dbg;
    }
    public static void display(String message) {
        getInstance().display(message);
    }
}
```

ضمن الـ Main :

أتعامل مع الـ SingletonDebugger بشكل مباشر وبالتالي ليسنا بحاجة إلى نقل المنقح ضمن كل البرنامج و إنما نقوم بتطبيق تابع display حسب نمط الـ debugger الذي مرر إلى SingletonDebugger.setInstance(dbg) كما في الكود التالي:

```
public class SolutionWithDebugger {
    public static void main(String[] args) throws Exception {
        S.o.p("Execution without debugger.");
        main();
        main("To Screen", new DebuggeToScreen());
        main("To File", new DebuggerToFile("dbg.txt"));
        main("In Window", new DebuggerInWindow());
        main("Withut Debugger", new WthoutDebugger());
    }

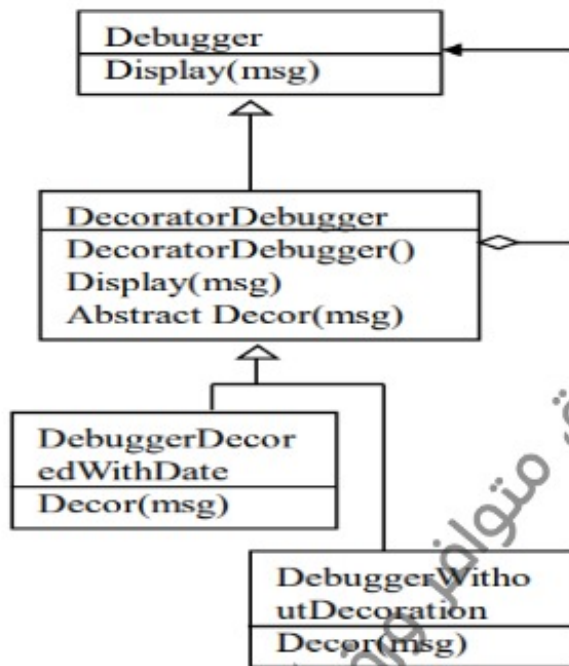
    public static void main(String title, Debugger dbg){
        S.o.p("Executin with Debugger " + title + ".");
        SingletonDebugger.setInstance( dbg );
        main();
    }

    public static void main() {
        SingletonDebugger.display("Begin of program");
        SingletonDebugger.display("Begin of for");
        for(int i=0; i<10; i++) {
            SingletonDebugger.display("Begin of step i=" + i);
            // ...
            SingletonDebugger.display("End of step i=" + i);
        }
        //...
    }
}
```

طريقة الـ Decorator:

في حال كان هناك متطلبات جديدة على البنية السابقة مثل إضافة تاريخ إلى رسة الـ debugger السابقة فإننا نقوم باستخدام الـ Decorator

لتحقيق هذه البنية ننشئ الكلاس DecoratorDebugger يرث من كلاس Debugger، ضمن الكلاس السابق نعرف الـ Display والـ decor (Abstract Method) من أجل استخدامه بالصفوف الجزئية مثل DebuggerWithoutDecoration , DebuggerDecoredWithDate (حيث قمنا بتمرير الـ Debugger إلى بواني الصفوف السابقة:

ضمن الـ Main :

```

public class SolutionWithDecoration {
    public static void main(String[] args) throws Exception
    {
        Debugger toScreen = new DebuggerToScreen();
        Debugger toFile = new DebuggerToFile("dbg.txt");
        Debugger inWindow = new DebuggerInWindow();

        SolutionWithDebugger.main("to screen with decoration",
            new DebuggerDecoredWithDate(toScreen) );

        SolutionWithDebugger.main("to file with decoration",
            new DebuggerDecoredWithDate(toFile) );

        SolutionWithDebugger.main("in window with decoration",
            new DebuggerDecoredWithDate(inWindow) );
    }
}
  
```

يمكن حل مثال الـ debugger عن طريق Composite Pattern لكن لم يقد الدكتور بشرحها.

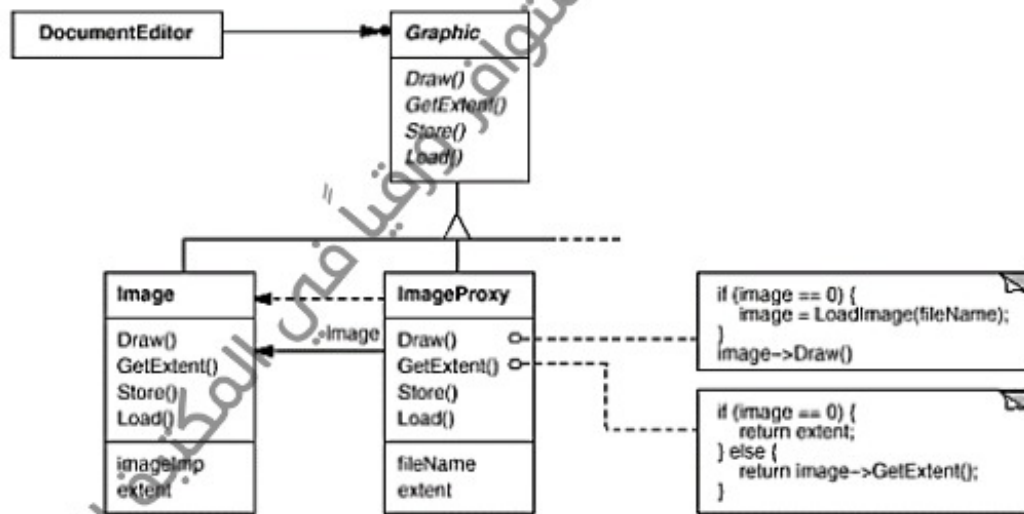
Proxy

تتوضح فكرته في مادة النظم الموزعة وعند شرح middleware.

مثال: Internet explorer

- قديماً كان بطيء، لأنه لا يمكنه تحميل الصفحة التي تحوي صور حتى تصل جميع المحتويات وبالتالي الصفحة التي تحتوي صور تأخذ وقت كبير.
- ثاني تطوير عليه كان بعرض الصفحة بشكل طبيعي ولكن عند وصول الصورة نضعها بمكانها، مثلاً أنت تقرأ النص بدون الصور، فتصل الصورة بعد تحميلها فيتبعثر ويتغير مكان النص الذي كنت تقرأه.
- والجيل الثالث من المتصفحات (ليس قديم جداً) عند تحميل صفحة وفيها صور والانترنت بطيء، يحجز في الصفحة إطار للصورة والنص يكتب طبيعي وعند وصول الصورة توضع في مكانها ← هنا استخدم ال Proxy.

على الانترنت تم تطوير هذه النقطة: ضمن ال webserver عند إرسال صفحة مع صور يرى الصور ويرسل عوضاً عنها اغراض Proxy (بديل) يمكنك معرفة معلومات بسيطة عن الصور (height,width) وبالتالي المتصفح يحجز مكان خاص بها ((كانت object ينوب عن object إلى حد معين)).

مثال:استخدامه:

1. Remote proxy:

يقدم ممثل محلي عن غرض موجود في مكان آخر (سيرفر آخر مثلاً) ، ضمن middleware (webserver ضمنها) وهو ليس غرضي التوجه.

أول برمجية ظهرت عام 1989 قبل ذلك عند كتابة تطبيق شبكي نستخدم TCP/UDP وكان يأخذ وقت طويل لأنه يتطلب فحص كبير وكان موجود الفكر الغرضي التوجه (استقر 1970).

Object middleware: الهدف الأساسي الخاص به أن نستطيع برمجة تطبيقات شبكية مثل البرمجة غرضية التوجه المحلية (عدم تخيل أن الغرض بعيد).

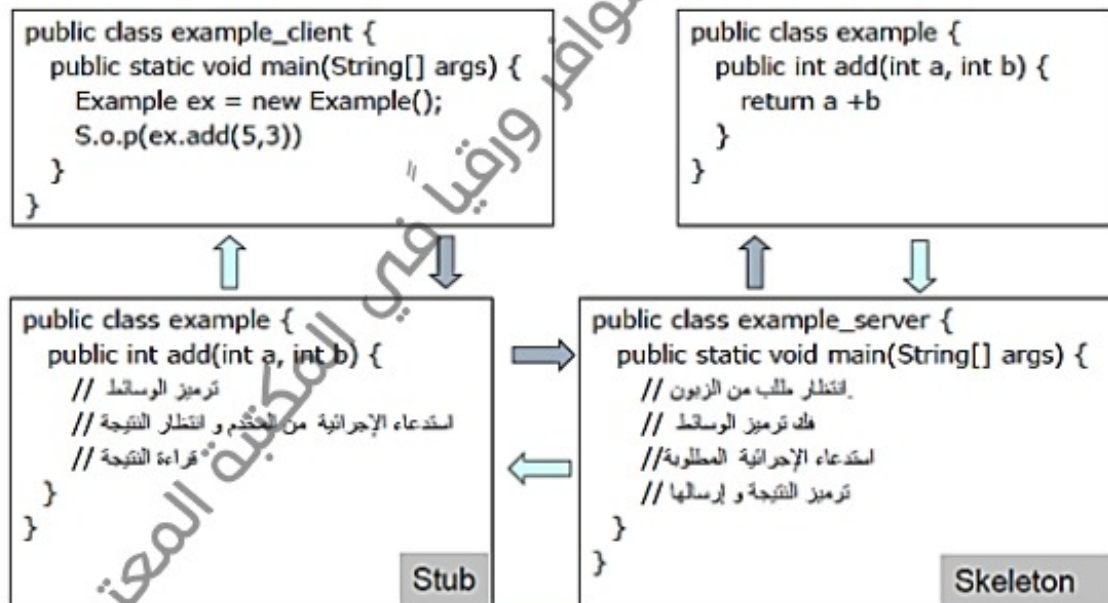
مثال : Stub & Skelton

لدينا صف `example_client` داخله التنفيذ `main` نعرف كائن من نوع `example` ومن ثم نستدعي `print(ex.add(5,3))` أي تم استدعاء `add` وليتم ترجمته يجب ان يكون لدينا على نفس الجهاز صف اسمه `example` يحوي `add` (هو `stub` وليس الصف الحقيقي لأن الصف الحقيقي موجود على مخدم خارجي) ولكن ضمن التنفيذ الخاص بتابع `add` تم ترميز الوسائط وإرسالها إلى المخدم وقراءة النتيجة من المخدم بدلاً من القيام بعملية الجمع بنفسه، وهو يتعامل مع ال `Skeleton`.

ال `Skeleton` هو ال `server`، يحوي صف `example_server`، ضمن ال `main` يقوم بانتظار طلب الزبون ثم فك ترميز الوسائط التي جاءت إليه `TCP/UDP` ثم استدعاء الإجرائية `add` ضمن الصف الحقيقي `example` الذي يحوي العملية الحقيقية ويمكنه استدعاءه لأنهما على نفس الحاسب وبعد الاستدعاء يقوم بترميز النتيجة وإرسالها مرة أخرى لبرنامجنا.

في النهاية Stub & Skelton غير ظاهرين للمبرمج فعلياً

رأى المبرمج أن لدينا إجرائية `add` ضمن الصف `example` ضمن جهاز ثاني وأنا لذي `reference` منه وأقوم باستخدامه ولكن هذا ال `ref` هو عبارة عن `proxy`. وبالتالي `middleware` تقوم بتسهيل البرمجة ولا تدخل في التفاصيل وإنما تكون الأمور مجردة.

**فائدة ال remote proxy إخفاء تفاصيل الشبكة****2. Virtual Proxy:**

أحيانا يكون لدينا أغراض كبيرة جدا فبدلاً من تحميلها في الذاكرة مباشرة نقوم بتحميل بديل عنها بسيط يمكنه رد العميات الأساسية، في حال تم طلب عملية كبيرة نقوم بتحميل الأصلية من الذاكرة (تذكر ال `Virtual memory` التي تم شرحها في مادة البنيان).

3. Protection Proxy

مثلاً لدينا غرض وله اتصال بال database (ونحن نعلم أن الناس يمكنها الوصول للأغراض عن طريق الشبكة) وبالتالي لعمل درجة حماية أكبر أقوم بعمل Proxy لحمايته أو تطبيق access control عليه، وعند حدوث خرق يتم الوصول لل Proxy ولا يمكنه الوصول لل database وبالتالي تم الفصل بطبقات.

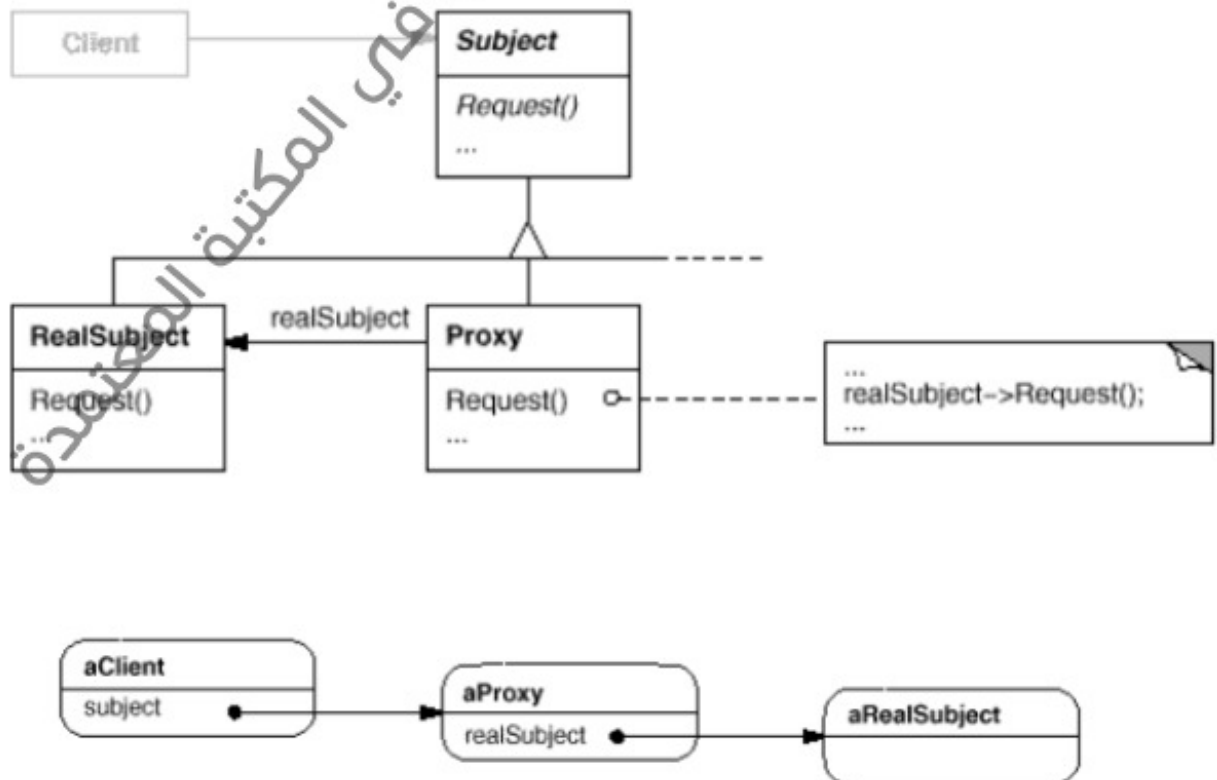
4. Smart reference

يتم شرحها وفهمها بشكل واضح في مادة النظم الموزعة. عبارة عن غرض موجود بكل بيئة يقوم بعد الأغراض ال Proxy وال garbage collector ضمن خوارزميته، يرى أنه في حال كان هنالك remote ref ولا أحد يستخدمه يتم حذفه، ولكن هنالك مشكلة أنه في حال حذف ref قام بإرسال رسالة فعند وصولها يعطي خطأ null reference نقوم بحلها عن طريق ال snapshot لحالة النظام لكل المواقع الموزعة ونضيف عليه أنها تأخذ حالة القنوات إذا كان هنالك remote ref على الشبكة نستطيع الوصول إليه وبالتالي لا يمكن حذف smart ref إذا كان يقوم بعملية.

ملاحظة

proxy هو الذي خلق ال middleware ويتم استخدام هذا ال design pattern ضمنه بشكل كبير. إحدى استخدامات ال proxy هو في المتطلبات غير الوظيفية (Usability)

بنية ال Proxy:



Adapter

المشكلة: عندما يكون لدينا نظامان: نظام خاص بي و نظام آخر أحتاج لربطهما.
فلدينا حلان:

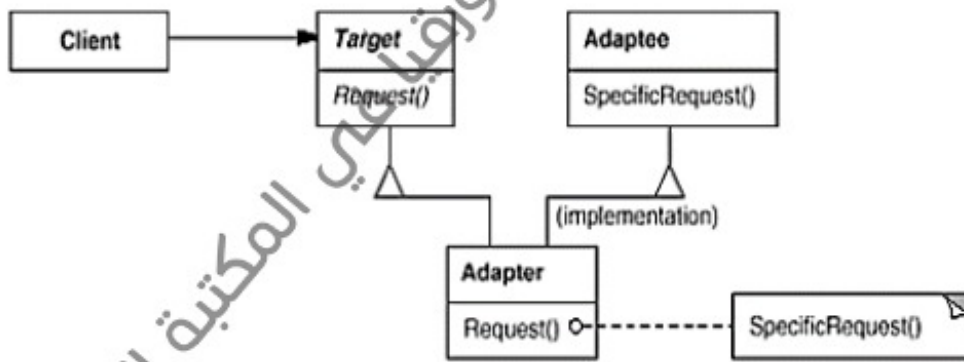
1. إما نقوم بتغيير النظام الآخر (interface) الخاصة به لجعله يتوافق مع نظامي.
 2. أو أن نقوم بتغيير النظام الخاص بنا.
- أو نأخذ الحل الثالث: بخلق نظام وسيط بينهما وهو الحل الذي يحافظ على التصميم.

ال Adapter له بنية: class pattern أو object pattern.

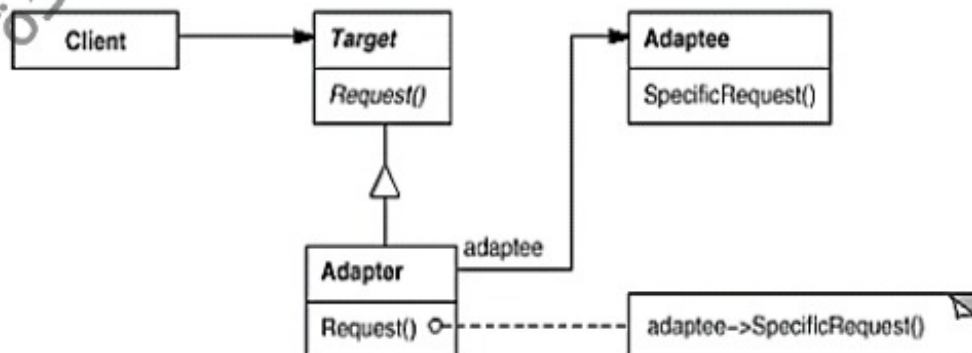
بنية ال class: تعتمد على العمليات بين الصفوف (classes) (الوراثة) حيث يرث ال Adapter من كلا الصنفين ويعيد كتابة العملية request باستخدام العملية specificRequest من الصف الآخر، وهذه الطريقة لا تناسب لغة ذات وراثة بسيطة مثل Java لا يمكن الوراثة إلا من صف واحد فقط.

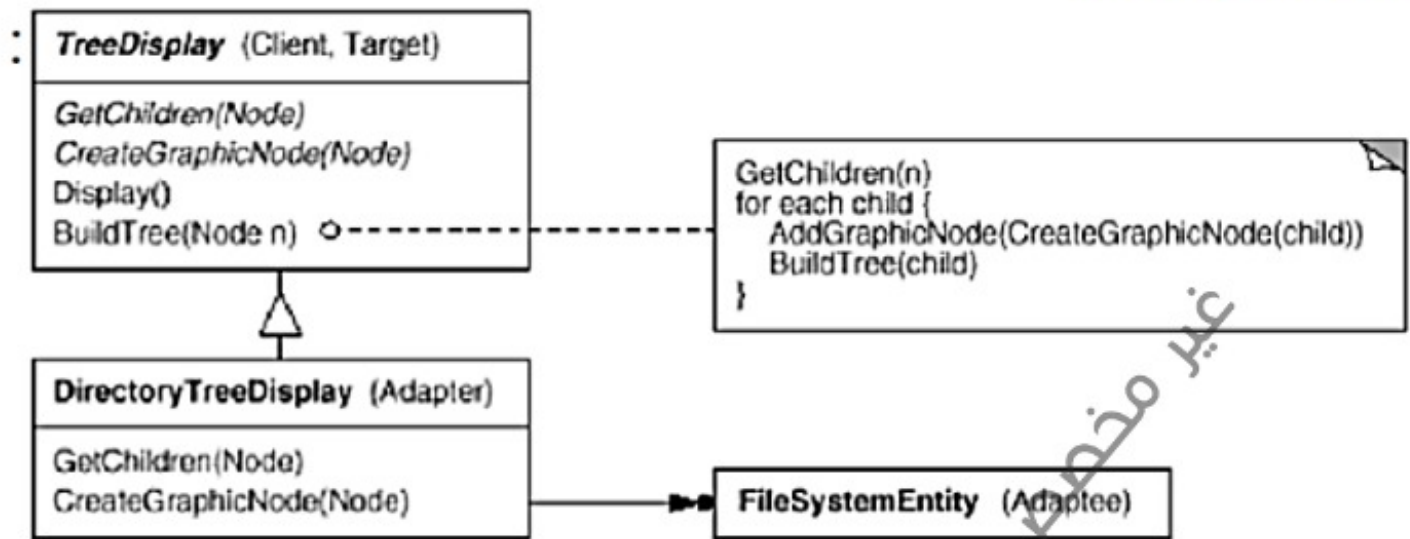
بنية ال object: تعتمد على العلاقات بين الأغراض objects (ال composition) حيث يرث ال Adapter من الصف الذي يريد استخدام عمليات الصف الآخر بينما يضم مرجع على غرض من الصف الثاني ويعيد كتابة عملية الصف الموروث request باستخدام عملية الغرض المضموم (المحتوى) specificRequest().

Class Adapter pattern



Object Adapter pattern





لدينا بنية هرمية لرسم الأشجار وأحتاج هذه البنية لرسم شجرة ال XML الخاصة بالملفات بحيث أنشأت لتكون reusable.

فيكون أساس بنائها:



- Get Children (node)
- Create Graphic Node (node)
- Build Tree (node)

يتم تنفيذ الأخيرة بشكل عودي إلى أن يتم رسم الشجرة

- نريد استخدام File system entity وهي adaptee ويكون الاستخدام بعمل Adapter نسميه مثلاً DirectoryTreeDisplay ونورثه من TreeDisplay ويصغر غرض من FileSystemEntity (composition).
- ونقوم بعمل Override لل `GetChildren (Node)` : نستدعي ضمنه `getFiles` على الغرض من نظام الملفات، ونكون بهذه الطريقة حافظنا على كلا المكتبتين دون تعديل وجعلناهما تعملان معاً.

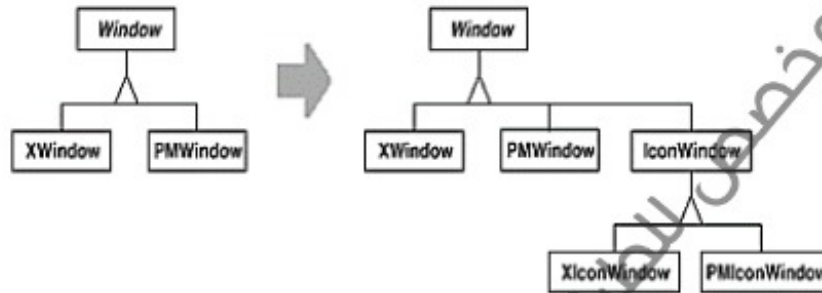
وبالتالي لم يتم تشويه الكود الأصلي وإنما يتم الوراثة ومن ثم صياغته فيما يناسب النظام. نمط عملية التعديل على ال method التي لا تناسب النظام يجب عمل وراثة ومن ثم override ال method التي لا توافق النظام.

Bridge

يسمح بالفصل بين هرمية التجريد وهرمية التنفيذ كي تتمكن كل هرمية من النمو بشكل مستقل. يعتمد بشكل رئيسي على إيجاد رابط بين التجريد والتنفيذ (يجب إيجاد هذا الرابط كي تنجح عملية الفصل).

مشكلة: لدينا بنية هرمية مثل window (مفهوم)

لدينا أنواع منها Xwindow و PMwindow وبلحظة معينة يأتي صف منطقي جديد (مفهوم) وهو IconWindow ونريد توزيعه للنوعين فتصبح الهرمية بالشكل التالي:



وبهذه الحالة تم الخلط بين الشيء المنطقي والذي هو المفهوم (نافذة بلا icon و نافذة مع icon) والشيء المادي والذي هو التنفيذ.

ويكون الحل: ضمن الـ Bridge هو يقط بين الهرمية المنطقية وهرمية التنفيذ.

ضمن window class لدينا ميثودين ساسيين

Draw Text();
Draw Rect();

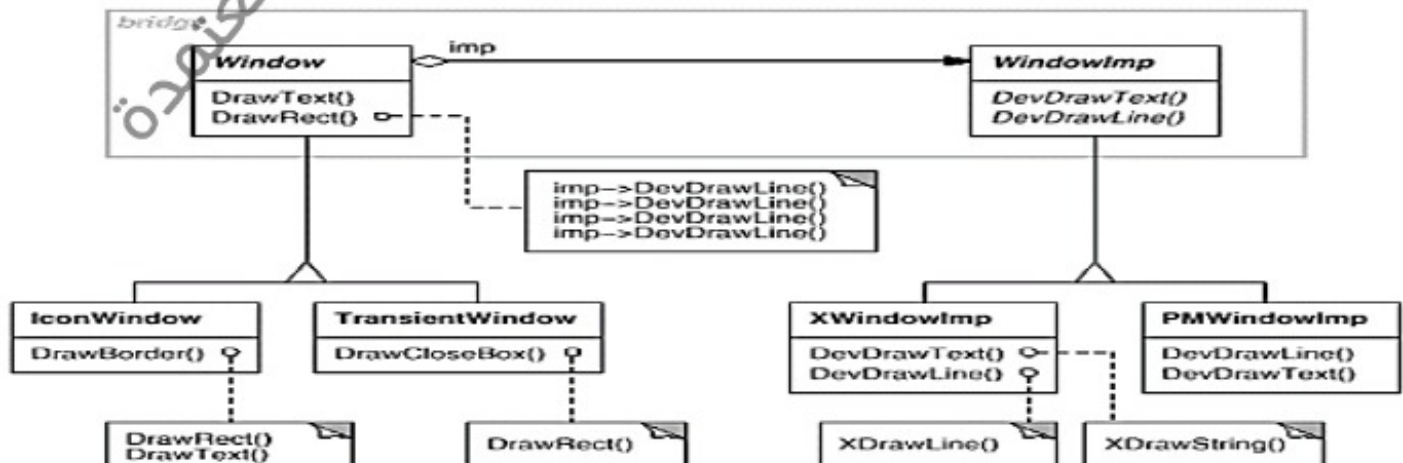
نقوم بعمل تجريد لهم:

Draw Text();
Draw Line();



وأنا أعلم أن Xwindow سيكون لديها draw text, draw line ونفس الأمر عند PMwindow طبعاً مع اختلاف في طريقة البناء.

ولذلك يبقى التنفيذ (المخطط الأول) الذي يبين التنفيذ، أي فيزيائياً نحن نلاحظ وجود Xwindow, PMwindow



الفكرة باختصار لدي

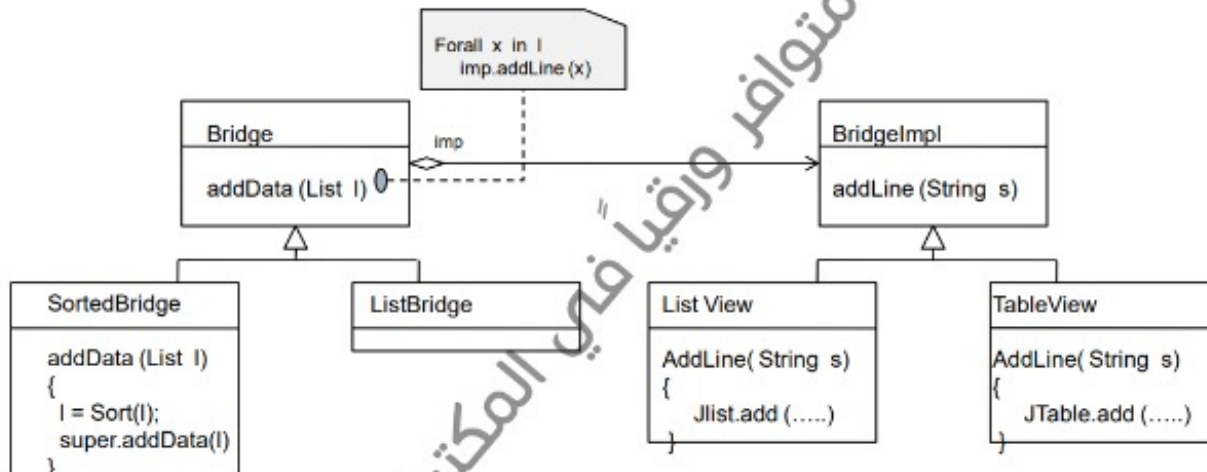
الجسر يصل بين مفهوم التنفيذ والمفهوم المنطقي (التجريد) وذلك عن طريق الComposition

مثال:

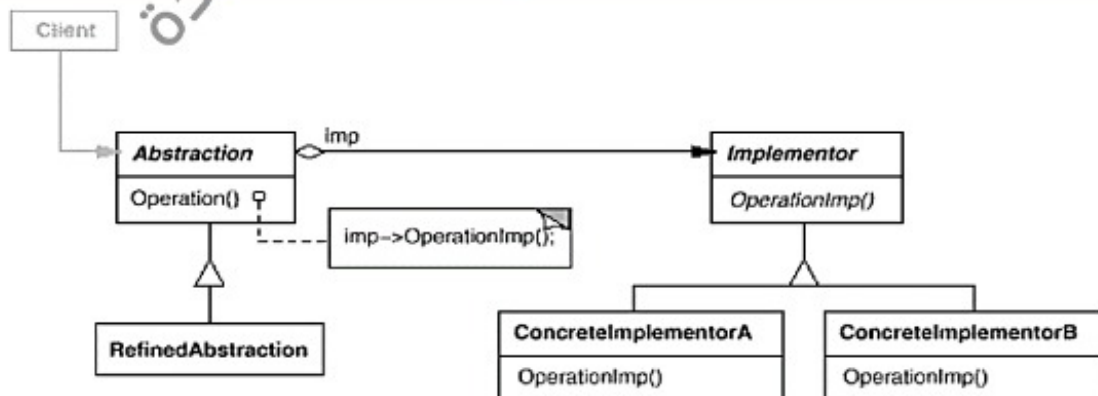
لدينا TableView وهي طريقة عرض على JTable ولدينا صف آخر وهو List view وتعتمد على List وكلاهما ينفذان addLine وبالتالي توضع ضمن صف أب وهو BridgeImp. الجانب المنطقي لدينا قائمة وقائمة مفروزة كلاهما يستخدمان addData وبالتالي يتم وضعها ضمن Bridge (صف أب) ولكن ضمن القائمة المفروزة نقوم بعمل إعادة بناء للـ addData لتقوم بالفرز ومن ثم يتم تنفيذ الـ addData عند الأب.

وبالتالي تم فصل ما هو متعلق بالبناء وما هو متعلق بالمفاهيم.

هكذا يمكن إدخال التنفيذ بالمفاهيم مع الحفاظ على هرمية صحيحة، وعند الإضافة يتم إضافة دون التعديل واستخدام التنفيذ بسهولة:



ملاحظة: حالات ورود حالة مثل Bridge ليست كبيرة بينما composite مع decorator هي كبيرة، أيضا حالات ورود factory method كبيرة (هناك نسب استخدام يجب الاطلاع عليها) والـ Adapter أيضا يستخدم كثيرا لكن دون الانتباه لاستخدامه (كونه منطقي).

بنية ال Bridge:

Flyweight

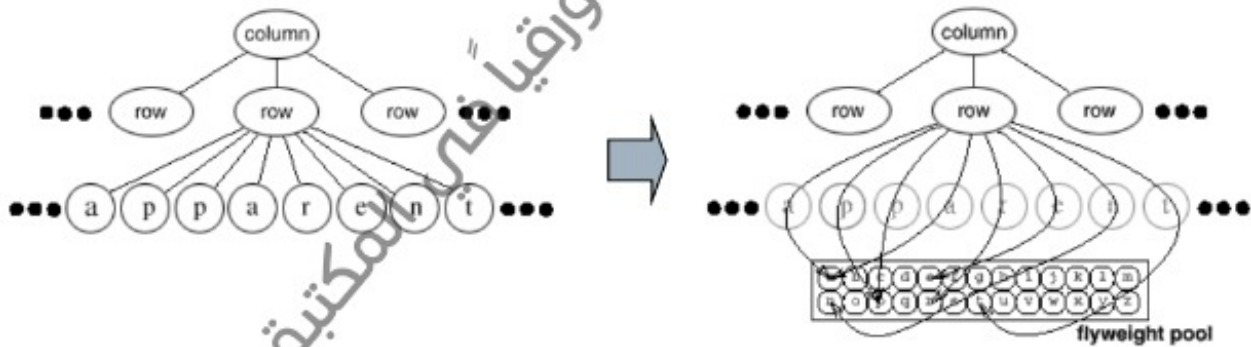
على فرض نريد القيام بـ object editor text يعني كل حرف يمثل بـ object وفيه مواصفات معينة (صورة، لون ..) فإذا كان لدينا نص من مئات الأسطر أي من مئات الآلاف من المحارف وبالتالي لدينا عدد هائل من الكائنات داخل الذاكرة، وبالتالي يجب أن تقوم بفصل الـ object ضمن البرنامج من ناحيتين الأولى المعلومات الثابتة والثانية المعلومات المتغيرة.

هذه الأحرف مشتركة بعدة واصفات (مثال شكل حرف a نفسه لا يتغير) فمن غير المعقول أن نقوم بإنشاء كائن عند المرور بكل كلمة تحتوي على الحرف a.

وبالتالي الحل الأفضل أن نقوم بإنشاء كائن واحد في الذاكرة ونولد (reference) مرجع خاص عند المرور بكل حرف a. ولكن هنالك معلومات متعلقة بالسياق (رقم السطر، رقم العمود، ...) ومعلومات ثابتة مثل صورة الحرف (هي التي تشغل الحجم الكبير).

ملاحظة: الخط المائل هو صورة عن الحرف بحيث السياق سيقوم بتحديد أي صورة يحتاجها لرسم الحرف

وبالتالي نقوم بعمل pool من الـ objects وعند كتابة كلمة ألقى نظرة على الـ pool وفي حال لم أجد object من الحرف الجديد المكتوب ضمن الكلمة أقوم بإنشائه وإضافته للـ pool ونقوم بعمل ref له ضمن الكلمة وأضيف معلومات السياق.

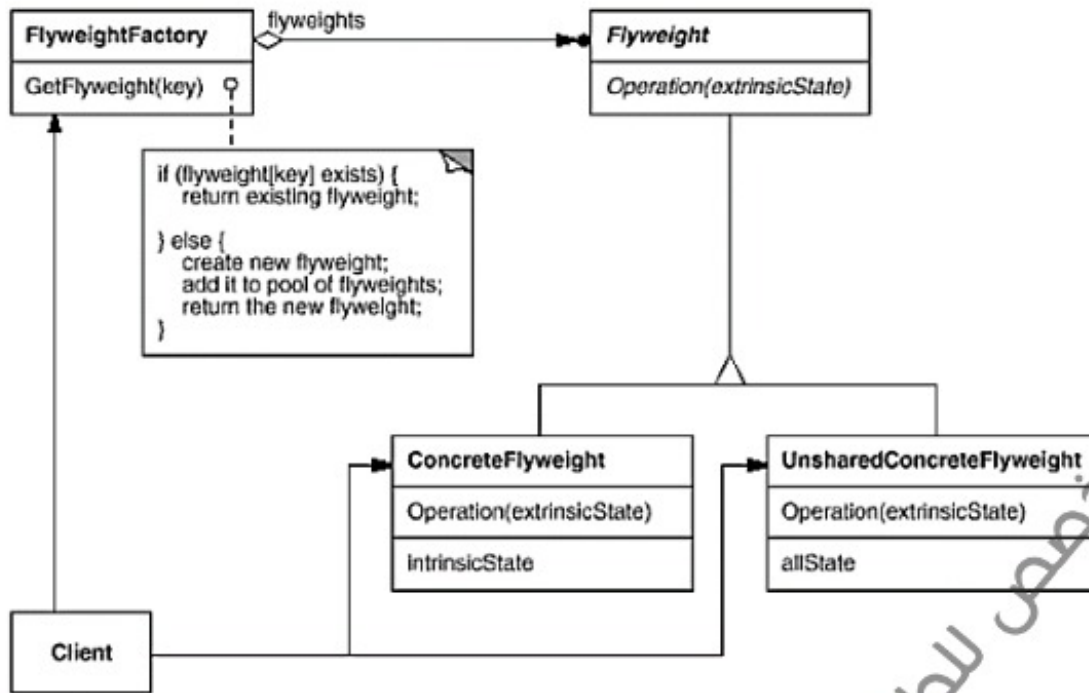


نتيجة : يتألف هذا التصميم من قسمين :

- Concrete flyweight وتحتوي الصفات المشتركة.
- Unsaved concrete flyweight: تحتوي معلومات السياق.

استخدامها:

مسائل امتحان: مثال لدينا برنامج للآليات في سورية فإن مواصفات السيارات معروفة، مثلاً سيارة Lada معروفة المواصفات (كم مقعد تمتلك ، استطاعة المحرك ، السعة) وعلى فرض لدينا مئة ألف سيارة هنا ننشئ غرض واحد ونقوم بعمل ref يحوي اسم المالك ، اللون ، الرقم....



يُفضل دراسة أسئلة الدورات المتعلقة بـ Design Patterns للتدريب والتعلم أكثر.

Behavioural patterns

وهي تحل مشاكل الخوارزميات والسلوك بين الأغراض ويتأخذ في هذه المحاضرة نوعين ضمنه:

1. Chain of Responsibility.

2. Template Method.

Chain of responsibility

مثال: ضمن الجافا exception نظام الاستثناءات هو نظام تسامح مع الأخطاء، إذا تم ارتكاب خطأ لن تكون نهاية البرنامج الخاص بك، إنما هناك مجال للإصلاح.

لدينا حالتان: لإلقاء الاستثناء throw ويمكنني استخدام throws للسماح له بالمرور.

لماذا استخدم الحاليتين

استخدم throws لأنني لا يمكنني أن أعالج (ليس لدي المعرفة لأعالج) مثل class overflow هو لا يعرف ماذا سيفعل في حال هذا الصف ولكن من يقوم باستخدامه ، لاحظ عند الإضافة ال overflow وبالتالي يقوم بخلق stack ثاني ويضيف عناصره. Stack مهمتي فقط وضع عناصر فيه واعيدها عند الطلب والذي يلاحظ ال overflow هو من يستخدم هذه ال stack.

Throw (exception) تعني لدي مشكلة وغير قادر على حلها.

معلومة : عند إنشاء ال java كانت design pattern مستقرة تماما وتم تطبيق جميع مفاهيمها ضمن هذه اللغة.

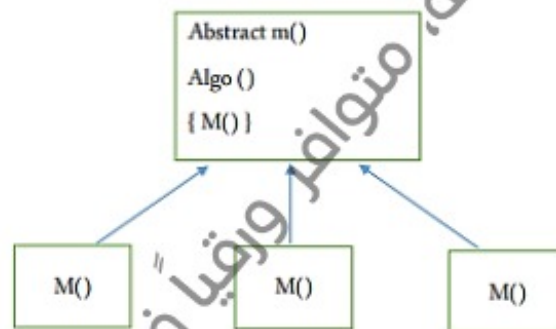
Template method

بسيط جدا ومستخدم بكثرة، يستخدم عندما يكون لدينا خوارزمية مؤلفة من مجموعة من الخطوات تنفذها يختلف حسب الـ subclass، كونها خوارزمية عامة لا داعي لإعادة كتابتها عند كل صنف من الاصناف وإنما نقوم ببناؤها وكتابتها في مكان واحد. الخطوة التي تكون متغيرة نقوم ببناؤها على اساس abstract method و subclasses نقوم بإعادة كتابتها بالطريقة المناسبة لها.

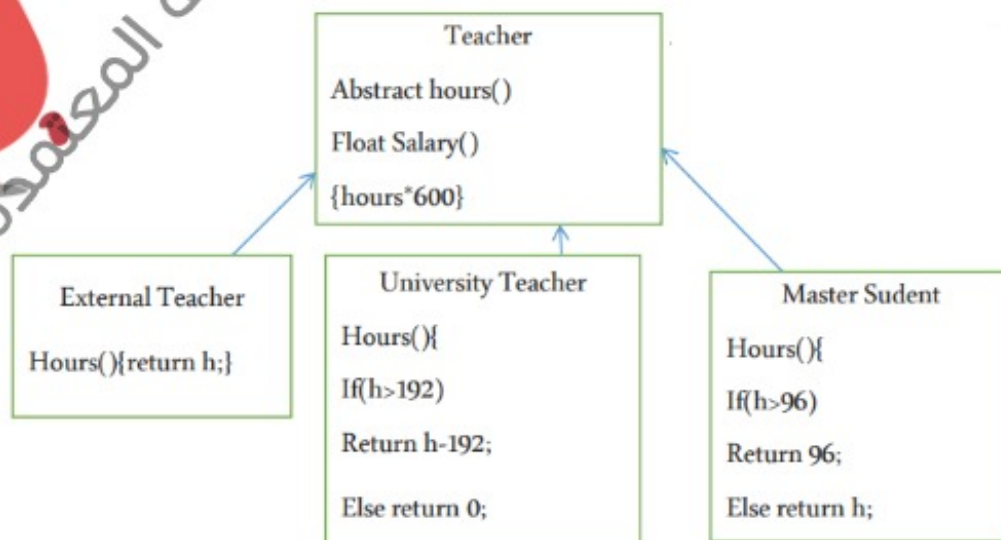
مثال: اساتذة الجامعة أعطى طريقة حساب الساعات الإضافية الخاصة بهم.

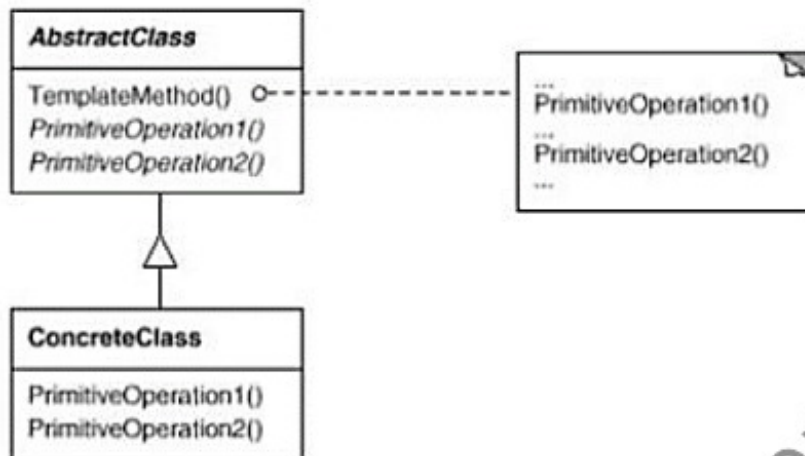
لدي ثلاث أنواع من الاساتذة:

- extern teacher : كل ساعات تدريسهم تعتبر ساعات إضافية.
- university teacher : ساعات التدريس التي تتجاوز حد ال 192 ساعة تعتبر ساعات إضافية.
- master student : طالب دراسات عليا: كل ساعاتهم إضافية بحيث لا تتجاوز 96 ساعة، له حد وذلك كي لا يصرف وقته في التدريس.



وخوارزمتنا تكون بحيث تضرب 600 بعدد الساعات.
حساب الساعات هي المشكلة لأن الناس (الأساتذة) أنواع





النهاية